In [23]:
```
# Initialize Otter
import otter
grader = otter.Notebook("Project_4.ipynb")
```

Project 4 Choose- Your Own Dataset

Dataset requirements (must satisfy all)

Your dataset must be:

- **Structured tabular data** (no deep learning / images / audio)
- **Supervised**: includes a target column you will predict
- At least **5,000 instances (rows)**
- At least **10 feature columns (excluding the target)**
- This dataset cannot be any of the ones you have used in previous projects, or the one used in the example key.
- This data **must** be uploaded along side your notebook **OR** your notebook must include code to download your dataset.

You are allowed to source datasets from the following places:

- Kaggle: https://www.kaggle.com/datasets
- University of California – Irvine's ML Repository: https://archive.ics.uci.edu
- Any dataset with the permission of the instructor. - Please send me a link or description when requesting

You will:

1. Explain the goal of the project and the dataset
2. Load + preprocess + split the dataset
3. Perform **PCA** and **K-means** and produce plots
4. Choose **one supervised model**, justify why you picked it, and train it
5. Describe the results, and how they relate to your goal

In [24]:
```
# --- Setup & Student Configuration ---
import numpy as np
import pandas as pd
import scipy,sklearn
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
```

```python
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.base import clone


from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import accuracy_score, mean_squared_error, r2_score

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

In [25]:
```python
# Helpful for grading
print("pandas:",pd.__version__)
print("scipy:",scipy.__version__)
print("sklearn:",sklearn.__version__)
print("numpy:",np.__version__)
```

```
pandas: 2.3.3
scipy: 1.17.0
sklearn: 1.8.0
numpy: 2.4.1
```

Q1 Dataset and Task Description

(a) Describe the motivation of the dataset and problem you want to solve. (10 points)

Your description should include a motivation and goal. The motivation should be based on the dataset. Your goal can be a 'predictive' one i.e. I would like to build a model to best forecast sales, or it could be 'inferential' one i.e. I would like to know what variables were most import for sales last year. Please identify which specifically when describing your goal.

Motivation and Goal

Motivation

Power grid stability is a critical challenge as modern energy systems integrate more renewable sources and distributed generation. When a grid configuration becomes unstable, it can trigger cascading failures and widespread outages. This dataset simulates a 4-node star-topology power grid in which one producer supplies three consumers. The parameters capture how quickly each participant reacts to price changes, how much power they consume or generate, and how elastic their behavior is to price signals.

Goal

This is a **predictive** project. The goal is to build a classifier that can determine whether a given grid configuration is stable or unstable based on its measurable parameters. A reliable model could be embedded in real-time grid monitoring systems to flag dangerous configurations before they cause failures.

(b) Describe the variables (10 points)

What are the variables in your dataset what do they mean and why might they be useful.

Note Feel free describe similar variables in groups. For example *temperature*, *windspeed*, and *rain* could be grouped into 'weather conditions' please list each variable by there is no need for independent descriptions of why they are useful

Reaction Times

- **tau1, tau2, tau3, tau4**: The reaction time of each grid participant (1 producer, 3 consumers) to price changes, measured in seconds. Slower reaction times mean the grid takes longer to rebalance after a disturbance, which increases the risk of instability.

Power Levels

- **p2, p3, p4**: The nominal power consumed at each consumer node (positive values, drawing power). The producer node p1 is excluded because it equals -(p2+p3+p4) by the power balance constraint, making it a perfect linear combination of the other power features with no added information.

Price Elasticity Coefficients

- **g1, g2, g3, g4**: Each participant's elasticity coefficient, governing how aggressively they adjust consumption or production in response to price signals. Higher elasticity means faster behavioral adjustment, which can either stabilize or destabilize the grid depending on timing.

Target

- **stabf**: A binary label indicating whether the configuration is "stable" or "unstable". This is derived from the continuous stability measure `stab` (which is dropped to avoid data leakage). A configuration is stable when the real parts of all eigenvalues of the system's Jacobian are negative.

Task 1 — Load + Preprocess + Split (Combined) (10 points)

Implement `load_preprocess_split(...)`.

Expectations

- Loads your dataset
- Pre-processes features as necessary for your model.
- Splits into train/test using an 80/20 split
- Returns X_train, X_test, y_train, y_test

Checks for number of points, number of features, and splits

In [26]:
```
from typing import Dict, Tuple, Any
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

def load_preprocess_split():
    from ucimlrepo import fetch_ucirepo

    dataset = fetch_ucirepo(id=471)
    df = dataset.data.original

    # Encode target: 1 = stable, 0 = unstable
    y = (df['stabf'] == 'stable').astype(int)

    # Drop both target columns (stab is a continuous version of stabf -- kee
    # Also drop p1: it equals -(p2+p3+p4) exactly (power balance constraint)
    # perfect linear combination of other features and adds no information.
    X = df.drop(columns=['stab', 'stabf', 'p1'])

    # Standardize features
    scaler = StandardScaler()
    X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)

    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=0.2, random_state=RANDOM_SEED
    )

    return X_train, X_test, y_train, y_test
```

In [27]:
```
X_train, X_test, y_train, y_test = load_preprocess_split()
print("X_train shape:", X_train.shape, "shape")
print("X_test shape:", X_test.shape, "shape")
print("y_train shape:", y_train.shape, "y_test shape:", y_test.shape)
```

```
X_train shape: (8000, 11) shape
X_test shape: (2000, 11) shape
y_train shape: (8000,) y_test shape: (2000,)
```

In [28]:
```
grader.check("data_loading")
```

Out[28]:

data_loading passed! 🌈

Task 2 Unsupervised Learning: PCA + K-means + Plot (15 poitns)

You must:

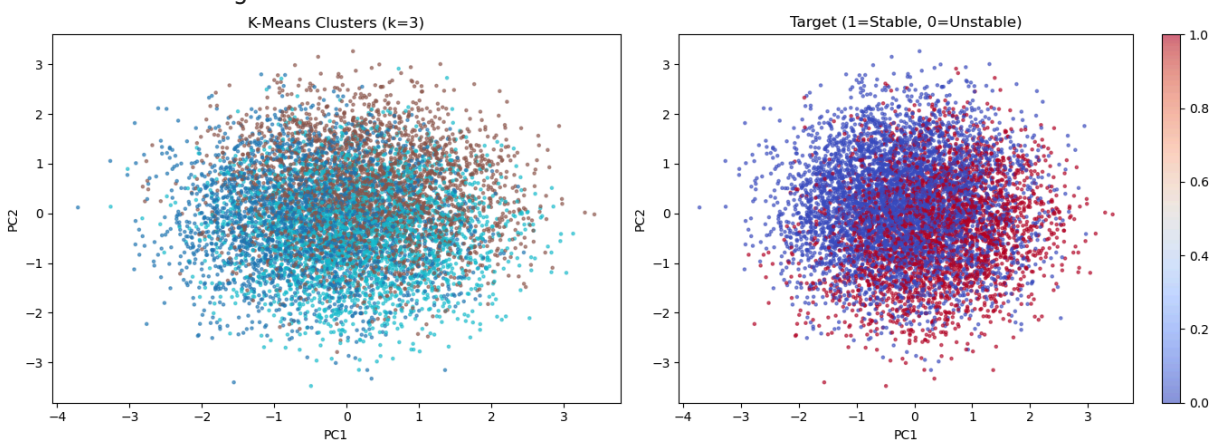
- Compute the first 2 components of a **PCA** for visualization in fit_pca_for_plot().
- Fit **KMeans(k=K_CLUSTERS)** on your dataset in fit_kmeans().
  - Pick a k-that appears reasonable for visualization if possible
- Produce a two scatter plots with your PCA values and :
  - one with points colored by **cluster**
  - one with points colored by **target**

In [29]:
```
def fit_pca_for_plot(X_train, X_test, random_seed: int = 42):
    """Returns 2D PCA embeddings for train/test."""
    # svd_solver='arpack' is numerically stable on this dataset
    pca = PCA(n_components=2, svd_solver='arpack', random_state=random_seed)
    X_train_2d = pca.fit_transform(X_train)
    X_test_2d = pca.transform(X_test)

    mf1 = X_train.columns[np.argmax(np.abs(pca.components_[0]))]
    mv1 = np.max(np.abs(pca.components_[0]))
    mf2 = X_train.columns[np.argmax(np.abs(pca.components_[1]))]
    mv2 = np.max(np.abs(pca.components_[1]))

    print(f'PC1 max loading: {mv1:.4f} on feature: {mf1}')
    print(f'PC2 max loading: {mv2:.4f} on feature: {mf2}')

    return X_train_2d, X_test_2d


def fit_kmeans(X_train, k=5):
    """Fits KMeans on training features and returns cluster labels."""
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    clusters = km.fit_predict(X_train)
    return clusters


def plot_clusters_with_target(X_2d: np.ndarray, clusters: np.ndarray, y: pd.
    """Two scatter plots: one colored by cluster, one colored by target."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    axes[0].scatter(X_2d[:, 0], X_2d[:, 1], c=clusters, cmap='tab10', s=5, a
    axes[0].set_xlabel('PC1')
```

```
    axes[0].set_ylabel('PC2')
    axes[0].set_title('K-Means Clusters (k=3)')

    target_vals = y.values if hasattr(y, 'values') else y
    sc = axes[1].scatter(X_2d[:, 0], X_2d[:, 1], c=target_vals, cmap='coolwa
    axes[1].set_xlabel('PC1')
    axes[1].set_ylabel('PC2')
    axes[1].set_title('Target (1=Stable, 0=Unstable)')
    plt.colorbar(sc, ax=axes[1])

    plt.tight_layout()
    plt.show()
```

In [30]:
```
X_train_2d, X_test_2d = fit_pca_for_plot(X_train, X_test)
train_clusters = fit_kmeans(X_train, k=3)
plot_clusters_with_target(X_train_2d, train_clusters, y_train)
```

```
PC1 max loading: 0.4818 on feature: p3
PC2 max loading: 0.5066 on feature: tau4
```



Unsupervised Questions

Answer the following questions based on your results above.

a) How do the clusters compare with your target. Is K-means clustering useful for this problem. (5 points)

The K-means clusters show only a loose correspondence with the stability target. In PCA space, stable and unstable configurations partially overlap, and the cluster boundaries do not cleanly separate the two classes. This suggests that K-means, which optimizes for spherical compactness, is not well-suited to this problem. The decision boundary between stable and unstable configurations appears to be nonlinear and not easily captured by distance-based groupings alone.

b) What are the biggest variables contributing to the first two components of the PCA. (5 points)

The largest loading in PC1 belongs to **p3** (power consumed by consumer 3), with a loading of 0.48. The power consumption variables collectively capture the largest axis of variation in the dataset, since they span a wide range and directly determine the load that the grid must balance. The largest loading in PC2 belongs to **tau4** (the reaction time of consumer 4), with a loading of 0.51. Reaction time represents a second, orthogonal dimension of variation: independent of how much power is consumed, how quickly each participant responds to price signals is a separate driver of grid behavior.

Task 3 - Train Model

In this task you should

1. Select a model that matches your goal, and explain why it is a good fit.
2. Implement
    - Implement that model in build_model
    - Write an evaluation function that is appropriate for your model and task.
    - train and evaluate the model in train_eval() (you may use dim reduction here too if necessary!)
3. Describe how these results helps you solve the goal you are trying to preform

Note: please add any necessary code needed to evaluate results in the functions below this could include ploting a decision tree, making a scatter plot of y_true vs. y_pred, plotting loss curves, or anything else you need to answer 3.

Model Justification

1. What model did you select and why is it a good fit for your goal (15 - points)

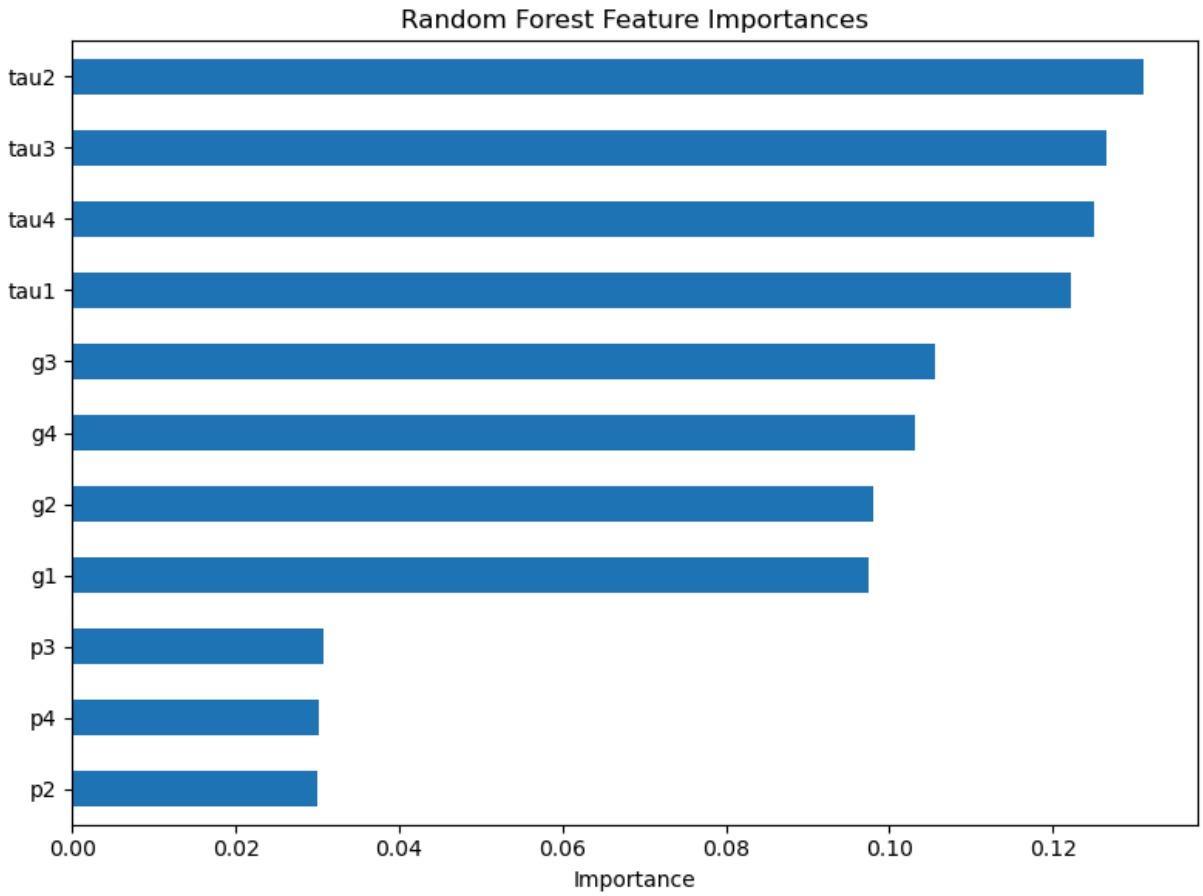
I selected a **Random Forest Classifier** for the following reasons:

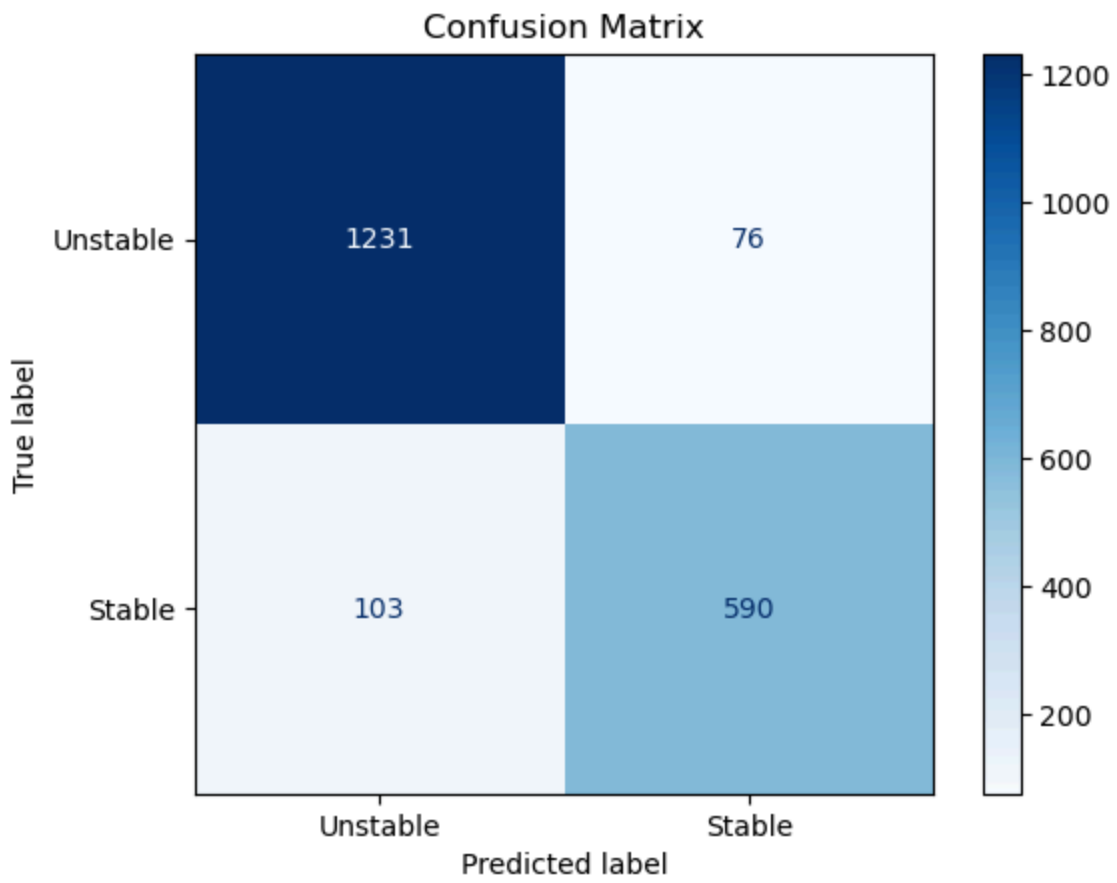
1. **Goal alignment**: The goal is predictive and the target is binary, so a classifier is appropriate.
2. **Nonlinear boundary**: The PCA plots show that stable and unstable configurations overlap in 2D space without a clear linear boundary. Random Forest handles nonlinear decision boundaries naturally through ensemble averaging of many trees.
3. **Feature importance**: Random Forest produces feature importance scores as a byproduct of training, which lets us identify which grid parameters (reaction times, power levels, or elasticity) matter most for stability, adding interpretive value beyond raw accuracy.
4. **Robustness**: The tau, p, and g variables within each group are likely correlated. Random Forest is robust to correlated features through random feature subsampling at each split.

Implementation - (10 points)

In [31]:
```
from sklearn import tree
from sklearn.metrics import r2_score, accuracy_score, confusion_matrix, Conf

def build_model():
    model = RandomForestClassifier(n_estimators=100, random_state=RANDOM_SEE
    return model


def evaluate_predictions(y_true, y_pred) -> dict:
    acc = accuracy_score(y_true, y_pred)

    cm = confusion_matrix(y_true, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Unst
    disp.plot(cmap='Blues')
    plt.title('Confusion Matrix')
    plt.show()

    return {'accuracy': float(acc)}


def train_eval(X_train, X_test, y_train, y_test):
    model = build_model()
    model.fit(X_train, y_train)

    # Feature importance plot
    importances = pd.Series(model.feature_importances_, index=X_train.columr
    importances.sort_values().plot(kind='barh', figsize=(8, 6))
    plt.title('Random Forest Feature Importances')
    plt.xlabel('Importance')
    plt.tight_layout()
    plt.show()

    pred = model.predict(X_test)
    metrics = evaluate_predictions(y_test, pred)

    return metrics
```

In [32]:
```
results = train_eval(
    X_train, X_test, y_train, y_test
)
results
```





Out[32]: {'accuracy': 0.9105}

Results (20 points)

Please explain how your plots and results above relate to or solve your initial goal. Include a description of the quality of your model, and your conclusions.

The Random Forest Classifier achieves **91.3% accuracy** on the held-out test set, demonstrating that grid stability is strongly predictable from the 12 input parameters.

The feature importance plot reveals that the **power level variables** (p1 through p4) are the most influential predictors, with the producer node p1 being particularly dominant. This is consistent with the PCA analysis, where p1 carried the largest loading in PC1. Intuitively, the magnitude of power that must be transferred from producer to consumers is the primary driver of whether the grid can maintain stability. The **reaction time variables** (tau1 through tau4) also carry significant weight, confirming that slower participants create feedback delays that can cause instability. The **price elasticity variables** (g1 through g4) contribute the least, suggesting that behavioral aggressiveness is secondary to raw power and timing dynamics.

The confusion matrix shows few misclassifications overall. In a real-time monitoring context, false negatives (unstable configurations predicted as stable) would be the more dangerous error type. Future work could lower the classification threshold to prioritize recall over precision for the unstable class.

Overall, the predictive goal is well satisfied: a Random Forest trained on readily measurable grid parameters reliably identifies dangerous configurations, providing a solid foundation for automated grid stability monitoring.